

Python 的零碎语法

石继楷 编写

2026 年 9 月 2 日

Contents

1 引言	2
2 基础输入与输出	2
2.1 print 的两个关键字参数	2
2.2 format 格式化输出	2
2.3 f-string 格式化输出	2
3 数值运算与类型转换	2
3.1 取整与舍入	2
3.2 进制转换与整除	3
3.3 提取各位数字	3
3.4 类型转换杂项	3
4 切片、布尔与逻辑运算	4
4.1 切片	4
4.2 布尔与逻辑运算	4
4.3 海象运算符	4
5 列表与序列操作	4
5.1 输入与拆分	4
5.2 列表增删与拼接	5
5.3 排序与逆序	5
5.4 查找与计数	5
5.5 去重	6
5.6 列表与字符串互转、解包	6
5.7 元组细节	7
6 循环与流程控制	7
6.1 range 循环与遍历	7
6.2 条件表达式	7
6.3 match-case	7
6.4 for...else 与 while...else	8
7 列表推导式、二维列表与 eval	8

8 经典算法片段	9
8.1 二分查找	9
8.2 选择排序	9
9 enumerate 枚举	9
10 函数	10
10.1 定义与调用、return	10
10.2 lambda 函数	10
10.3 增量赋值与可变性	10
10.4 默认值函数	11
10.5 可变参数	11
11 高级函数：zip、reduce、filter	11
12 异常处理	12
13 结语	12

1 引言

本教程旨在为正在学习 Python 的读者提供一份实用的常用功能指南，将零碎的语法点按主题归类整理。内容涵盖基础输入输出、数值运算与类型转换、切片与逻辑运算、列表与序列操作、循环与流程控制、列表推导式与二维列表、经典算法片段、函数与高级函数、异常处理等核心知识点。通过本教程，读者可以快速掌握 Python 编程中的常用技巧与最佳实践。

2 基础输入与输出

2.1 print 的两个关键字参数

print 函数支持两个常用的关键字参数：sep（分隔符，默认为空格）和 end（结束符，默认为换行）。

```
1 # 两个参数输出（关键字参数）sep=' ' / end=''
2 print("Hello", "World", sep="-", end="!")
3 # 输出：Hello-World!
```

2.2 format 格式化输出

使用 format 方法时，花括号中的数字表示参数位置，按顺序依次填入。

```
1 f1, f2, f3 = 1, 2, 1.5
2 # 依次填入：{2} 用 f3, {0} 用 f1, {1} 用 f2
3 s = '{2}与{0}的平均值是{1}'
4 print(s.format(f1, f2, f3))
```

2.3 f-string 格式化输出

f-string 是更直观的格式化方式，花括号内可直接写表达式，冒号后控制小数位数。

```
1 # fstring 方法
2 print(f"{f1}与{f2}的平均值为{(f1+f2)/2:0.2f}。")
3 a = f'{5.1:.2f}' # 输出 5.10
4 print(a)
```

3 数值运算与类型转换

3.1 取整与舍入

- round(): 四舍六入五成双。注意 round(3.5)=4, round(4.5)=4; round(4.5,2) 输出 4.5。
- 导入库: from math import floor, ceil。floor 向下取整, ceil 向上取整。
- int(2.3)=2 (直接截断小数部分)。

```

1 from math import floor, ceil
2 print(round(3.5))      # 4 (四舍六入五成双)
3 print(round(4.5))      # 4
4 print(round(4.5, 2))   # 4.5
5 print(int(2.3))        # 2
6 print(floor(2.9))      # 2, 向下取整
7 print(ceil(2.1))       # 3, 向上取整

```

3.2 进制转换与整除

```

1 # int('100', 2) 输出 2 进制的 100 对应的十进制
2 print(int('100', 2)) # 4
3 # 整除与负数
4 print(-9 // 2)        # -5 (向下取整)

```

3.3 提取各位数字

利用 `//10` (将数字删掉最后 1 位) 与 `%10` (取当前数最后一位) 可以提取各位数字。

```

1 i = 456
2 # 提取百位数字
3 a = i // 100      # 4
4 # 提取十位数字
5 a = (i // 10) % 10 # 5
6 # 提取个位数字
7 a = i % 10        # 6
8 # 小结: //10 将数字删掉最后 1 位; %10 取当前数最后一位

```

3.4 类型转换杂项

- `str()`: 把数字和 `True`、`False` 转成字符串。
- 参数区分大小写: 关键字 `True/False` 首字母必须大写。

```

1 print(str(100))      # '100', 数字转字符串
2 print(str(True))     # 'True', 布尔转字符串

```

4 切片、布尔与逻辑运算

4.1 切片

切片语法为 `a[start:stop:step]`: 开始取, 结束不取, 最后一个元素的索引为 `-1`。

```

1 a = [0, 1, 2, 3, 4]
2 # a[1:3:1] 从第二个元素开始，索引到第四个字符（不取），步长为 1
3 print(a[1:3:1])    # [1, 2]
4 print(a[-1])       # 4，最后一个为 -1

```

4.2 布尔与逻辑运算

- not 1 结果为 False。
- bool(list): 列表是否有元素，有则为 True，空列表为 False。
- and 找假、返回后一个: 2 and 3 得 3; 2 and 0 得 0。
- or 找真、返回前一个: 2 or 3 得 2; 0 or 2 得 2。
- 优先级: not 高于 and, and 高于 or。

```

1 print(not 1)          # False
2 print(bool([1]))     # True, 列表是否有元素
3 print(bool([]))      # False
4 # 找假/后
5 print(2 and 3)       # 3
6 print(2 and 0)       # 0
7 # 找真/前
8 print(2 or 3)        # 2
9 print(0 or 2)        # 2
10 # 优先级: not > and > or

```

4.3 海象运算符

(b:=3)>0: 先把 3 赋值给 b, 再判断 3>0。

```

1 if (b := 3) > 0:     # 把 3 赋值给 b, 再判断 3 > 0
2     print(b)         # 3

```

5 列表与序列操作

5.1 输入与拆分

```

1 # xxx.split() 依靠空格创建列表
2 s = "a b c"
3 lst = s.split()      # ['a', 'b', 'c']
4 print('abc'.split()) # 结果是 ['abc'], 无空格则不拆分
5
6 # 输入列表 (整数)
7 fs = list(map(int, input().split()))

```

```

8 # 输入列表（字符串）
9 fs = list(map(str, input().split()))
10 # 把输入字符串逐字符打散成列表
11 fs = list(str(input()))
12
13 # 打散字符串
14 print(list('abcde'))      # ['a', 'b', 'c', 'd', 'e']

```

5.2 列表增删与拼接

```

1 fs = [1, 2, 3]
2 fs.append(1)          # 往列表 fs 里加元素
3 fs.pop(5)             # 删 fs 索引 5 处的元素
4
5 fs1 = [1, 2]
6 fs2 = [3, 4]
7 fs1 += fs2            # 列表相加，fs1 变为 [1, 2, 3, 4]
8 print([1, 2] + [3, 4]) # [1, 2, 3, 4]
9 print([1, 2] * 3)     # [1, 2, 1, 2, 1, 2]
10
11 # 想要把第 i 个元素加入新列表
12 score = fs[i]
13 new_fs.append(score)

```

5.3 排序与逆序

```

1 fs = [3, 1, 2]
2 fs2 = fs[::-1]        # 逆序输出，不改变原列表
3 fs.reverse()          # 改变 fs，原地逆序
4 fs.sort(reverse=True) # 从大到小排序，fs 会变
5 lst = sorted(fs)      # lst 是排序结果，fs 不会变

```

5.4 查找与计数

```

1 fs = [1, 2, 3, 2]
2 # 序列的操作：成员判断
3 print(2 in fs)        # True
4 print(max(fs))        # 3，最大值
5
6 s = "hello"
7 print(s.find('l'))     # 返回位置 2，如果没有则返回 -1
8 print(s.rfind('l'))    # 从右往左找，返回正索引 3
9
10 # 列表计数：lst.count(num)

```

```

11 print(fs.count(2))          # 2, 找 a 出现的次数
12
13 # 计算列表里面字符串的最大长度
14 max_length = max(len(word) for word in fs)
15
16 # 一个一个单词对比
17 words = s.split()
18 res = []
19 target = "l"
20 for word in words:
21     if word == target:
22         res.append(target)

```

5.5 去重

```

1 x = [1, 2, 2, 3]
2 y = set(x)          # 把列表 x 变成集合, 去重
3
4 # 有序 set, 只保留一个 (保持原顺序)
5 unique_char = []
6 for c in x:
7     if c not in unique_char:
8         unique_char.append(c)

```

5.6 列表与字符串互转、解包

注意 join 的元素必须是字符串。

```

1 result = ['a', 'b', 'c']
2 # 列表转字符串
3 print(" ".join(result))          # a b c
4 # 元素是数字时, 先用 map(str, ...) 转换
5 print(' '.join(map(str, [1, 2]))) # 1 2
6
7 word = "ABC"
8 print(word.lower())              # abc, 转小写
9
10 # 解包
11 lst = [1, 2, 3]
12 print(*lst, sep='\n')           # 解包后每个元素一行

```

5.7 元组细节

```

1 print(type((1)))          # <class 'int'>, (1) 是 int
2 print(type((1,)))         # <class 'tuple'>, (1,) 是元组

```

```
3 print((1, 2) is (1, 2)) # 不一定对，要看地址
```

6 循环与流程控制

6.1 range 循环与遍历

```
1 # for i in range(1, 21): i 从 1 到 20 循环
2 for i in range(1, 21):
3     print(i)
4
5 # 遍历列表
6 fs = [1, 2, 3]
7 for num in fs:
8     print(num)
```

6.2 条件表达式

```
1 y = 2
2 # 条件表达式: 如果 y != 1 把 2 赋值给 x
3 x = 3 if y == 1 else 2
4 print(x) # 2
```

6.3 match-case

match-case 是 Python 3.10 及以上版本的结构匹配语法，case 1|2|3 表示同时匹配多个值，case _ 为默认分支。

```
1 today = 5
2 match today:
3     case 1 | 2 | 3: # 匹配多个值
4         print("前半周")
5     case 5:
6         print("周五")
7     case _: # 默认分支
8         print("其他")
```

6.4 for...else 与 while...else

for...else 和 while...else 是和 break 搭配的“善后”结构：循环未被 break 打断时才执行 else 块；也可以用 found 标志实现同样的善后。

```
1 fs = [1, 2, 3]
2 target = 4
3 found = False
```

```

4 for num in fs:
5     if num == target:
6         found = True
7         break
8     else:
9         # 循环没有被 break 打断时执行 (善后)
10    print("未找到目标值")
11    # 或者用 found 标志善后
12    if not found:
13        print("未找到目标值")

```

7 列表推导式、二维列表与 eval

列表推导式的通用形式为 `[exp(item) for item in oldList if condi(item)]`。

```

1 # 列表解析式 (列表推导式)
2 # [exp(item) for item in oldList if condi(item)]
3 def is_prime(n):
4     if n < 2:
5         return False
6     for i in range(2, int(n ** 0.5) + 1):
7         if n % i == 0:
8             return False
9     return True
10
11 # eg: 筛出 1000 以内的素数
12 ans = [i for i in range(1000) if is_prime(i)]
13
14 # 二维列表初始化
15 m, n = 3, 4
16 lst = [[0 for _ in range(m)] for _ in range(n)]
17
18 # eval(): 直接输入转可执行代码
19 print(eval("3 + 4"))    # 7

```

8 经典算法片段

8.1 二分查找

```

1 low = 0
2 high = len(lst)
3 found = False
4 while low < high:
5     mid = (low + high) // 2
6     if n == lst[mid]:

```

```

7 found = True
8 break
9 elif n < lst[mid]:
10 high = mid
11 else:
12 low = mid + 1

```

8.2 选择排序

```

1 for i in range(0, len(del_lst) - 1):
2     min_index = i
3     for j in range(i + 1, len(del_lst)):
4         if del_lst[min_index] > del_lst[j]:
5             min_index = j
6     del_lst[i], del_lst[min_index] = del_lst[min_index], del_lst[i]

```

9 enumerate 枚举

enumerate 可同时取得索引与元素，默认 start 为 0。

```

1 fruits = ["苹果", "香蕉", "橙子", "葡萄"]
2 for index, fruit in enumerate(fruits, start=1):
3     print(f"第 {index} 种水果是: {fruit}")
4 result = list(enumerate(fruits, start=1))
5 print(result)
6 # [(1, '苹果'), (2, '香蕉'), (3, '橙子'), (4, '葡萄')]
7 # 默认 start 为 0

```

10 函数

10.1 定义与调用、return

定义函数用 def，注意函数有没有 return：没有 return 时，函数返回 None。

```

1 def function(x, y):
2     return x + y
3
4 # 调用
5 b = function(1, 2)
6 print(b)    # 3
7 # 注意有没有 return：无 return 的函数返回 None

```

10.2 lambda 函数

lambda 是一个不用 def 的匿名函数，常用于排序的 key 参数。

```
1 a = lambda x, y: x + y
2 second = lambda x: x[1]    # 一个不用 def 的函数，返回第二个元素
3 lst = [[5, 6], [1, 2]]
4 # 根据 [5, 6] 的第二个元素排序
5 lst.sort(key=lambda x: x[1])
6 print(lst)    # [[1, 2], [5, 6]]
```

10.3 增量赋值与可变性

不可变对象（int、str）增量赋值时会重新复制（开辟新内存）；可变对象（如列表）则原地增量（不开辟新内存）。

```
1 def func1(x):
2     x += 100    # int 不可变，重新赋值，只改变 x 自己
3
4     a = 5
5     func1(a)
6     print(a)    # 5, a 不受影响
```

10.4 默认值函数

定义默认值函数时，应避免用可变对象（如列表）作为默认值。

```
1 def babble(words, times = 1, spaces = 0):
2     print((words + ' ' * spaces) * times)
3
4     babble('Hello')           # Hello
5     babble('Tiger', 3)        # TigerTigerTiger
6     babble("Three", 3, 2)     # Three Three Three
```

10.5 可变参数

*a 可以接收一个或多个位置参数，打包成元组；**a 接收带参数名的参数（关键字参数），打包成字典。

```
1 def my_sum(a, b, *c, **d):
2     print(a, b, c, d)
3
4     print(my_sum(1, 2, 3, 4, 5))
5     # a:1, b:2, c:(3, 4, 5), d: {}
6     print(my_sum(1, 2, 3, 4, 5, male = 6, femal = 7))
7     # a:1, b:2, c:(3, 4, 5), d: {'male': 6, 'femal': 7}
```

11 高级函数：zip、reduce、filter

```
1 from functools import reduce
2
3 # zip: 把多个可迭代对象打包
4 names = ["Alice", "Bob", "Charlie"]
5 math = [85, 92, 78]
6 english = [90, 88, 82]
7 zipped = zip(names, math, english)
8 # 转换为列表查看
9 result = list(zipped)
10 print(result)
11 # 输出: [('Alice', 85, 90), ('Bob', 92, 88), ('Charlie', 78, 82)]
12
13 # reduce() 累计
14 seq = [1, 2, 3, 4, 5]
15 total = reduce(lambda x, y: x + y, seq)
16 print(total)    # 15
17
18 # filter(): fun 必须返回 True/False (表达式)
19 lst = [1, 2, 3, 4, 5]
20 ans = list(filter(lambda x: x % 2 == 0, lst))
21 print(ans)      # [2, 4]
```

12 异常处理

使用 try...except 捕获并处理运行时错误，防止程序崩溃。

```
1 try:
2     result = 10 / 0
3 except Exception as e:
4     print(e)    # 打印异常信息
```

13 结语

本教程梳理了 Python 编程中最常用的零碎语法点与高级特性。掌握这些知识点，将有助于读者编写出更加简洁、高效且健壮的 Python 代码。建议读者在实际项目中多加练习，以巩固所学内容。

附录

本教程使用的模板如下，读者可以参考本文档的字体、标题样式、`lstlisting` 设置以及添加的宏包等。

```
\documentclass{article}
\usepackage{geometry}
\geometry{
  a4paper,
  left=2.5cm,
  right=2.5cm,
  top=2.5cm,
  bottom=2.5cm,
  headheight=13.6pt
}
% 最简单、最稳定的中文方案（自动适配系统，永不报错）
\usepackage[UTF8, scheme=plain]{ctex}
% 设置英文默认字体为 Times New Roman
\usepackage{fontspec}
\setmainfont{Times New Roman}
\usepackage{amsmath}
\usepackage{amssymb}
\usepackage{booktabs}
\usepackage{caption}
\usepackage{setspace}
\onehalfspacing
\setlength{\parskip}{4pt}
\usepackage{array}
\usepackage{listings}
\usepackage{graphicx}
\usepackage{enumitem}
\usepackage{subcaption}
\usepackage{multirow}
\usepackage{mathrsfs}
\usepackage{bm}
\usepackage{xcolor}
\usepackage{colortbl}
\usepackage{tikz}
\usepackage{tikz-3dplot}
\usetikzlibrary{
  arrows.meta,
  calc,
  patterns,
  decorations.pathmorphing,
  backgrounds,
  positioning,
  fit,
  shapes.geometric,
```

```

intersections,
through,
angles,
quotes,
plotmarks
}
%目录超链接
\usepackage{hyperref}
\hypersetup{
colorlinks=true,
linkcolor=black,
filecolor=magenta,
urlcolor=cyan,
}
\usepackage{tocloft}
\setlength{\cftbeforesecskip}{2ex}
\setlength{\cftbeforesubsecskip}{1ex}
\setlength{\cftbeforesubsubsecskip}{1ex}
% 定义代码颜色
\definecolor{codegreen}{rgb}{0,0.6,0}
\definecolor{NavyBlue}{RGB}{0,0,128}
\definecolor{PineGreen}{RGB}{1,121,111}
% 配置 listings 环境
\lstset{
language=Python,
tabsize=4,
captionpos=b,
numbers=left,
numbersep=1em,
sensitive=true,
showtabs=false,
frame=shadowbox,
breaklines=true,
keepspaces=true,
showspaces=false,
showstringspaces=false,
breakatwhitespace=false,
basicstyle=\ttfamily\normalsize,
keywordstyle=\color{NavyBlue},
commentstyle=\color{codegreen},
numberstyle=\color{black},
stringstyle=\color{PineGreen!90!black},
rulesepcolor=\color{red!20!green!20!blue!20},
escapeinside={/@}{@/},
}
\definecolor{LightGreen}{rgb}{0.9, 0.95, 0.9}
\definecolor{LightBlue}{RGB}{229, 238, 255}

```

```

\definecolor{LightPink}{RGB}{255, 229, 237}
\captionsetup[table]{
  justification=centering,
  labelsep=quad,
  font={bf}
}
\captionsetup[figure]{
  justification=centering,
  labelsep=quad,
  font={bf}
}
\captionsetup[subfigure]{
  font={bf}
}
\begin{document}
  ...
\end{document}

```